

AI / ML INTERNSHIP PROGRAM DEVIXO

SOLUTIONS

Task 04 End-to-End ML Application with Deployment

Version 1.0 approved

Prepared by <Muhammad Hamza Waseem>

Submission Date : 04-10-26

Customer Churn Prediction – Project Report	1
1. Introduction	2
2. Objective	2
3. Dataset Name	3
4. Dataset Source	3
5. Dataset Description.....	3
6. Dataset Size	3
7. Features/Columns Description.....	4
8. Methodology	5
8.1 Workflow	5
8.2 Preprocessing Pipeline.....	6
8.3 Models Trained.....	6
8.4 Model Results (before tuning)	7
8.5 Hyperparameter Tuning	7
8.6 Streamlit Application	8
8.7 FastAPI Service	11
8.8 Deployment	12
9. Data Loading.....	12
10. Data Cleaning.....	13
11. Exploratory Data Analysis (EDA)	13
12. Statistical Analysis.....	14
13. Data Visualizations	15
14. Code Screenshots	16
15. Output Screenshots.....	17
16. Graph Screenshots.....	18
17. Major Findings	20
18. At Least 10 Insights	21
19. Conclusion	23
20. GitHub Repository Link	24

Customer Churn Prediction – Project Report

1. Introduction

Customer churn is the loss of customers who stop using a company's product or service. For subscription-based businesses, keeping an existing customer is usually cheaper than acquiring a new one, so being able to predict which customers are likely to leave is very valuable.

This project builds a complete machine learning solution for churn prediction. It covers data loading, cleaning, exploratory data analysis (EDA), statistical analysis, training of four classification models, hyperparameter tuning of the best model, and deployment of the final model through two interfaces: an interactive Streamlit dashboard and a FastAPI REST service.

2. Objective

The main objective is to predict whether a customer will churn (1) or stay (0) based on their profile, usage, support and billing information. The specific goals are:

- Load, inspect and clean the customer churn dataset.
- Explore the data and identify the factors related to churn.

- Train and compare four machine learning models: Logistic Regression, Decision Tree, Random Forest and K-Nearest Neighbors (KNN).
- Improve the best model with hyperparameter tuning (RandomizedSearchCV).
- Save the trained pipeline and serve it through a Streamlit application and a FastAPI service.
- Provide prediction probability, feature importance, data visualization, model performance and multiple model selection as bonus features.

3. Dataset Name

Customer Churn Dataset (file: customer_churn_dataset-training-master.csv).

4. Dataset Source

The dataset is a publicly available customer churn dataset obtained from Kaggle.

5. Dataset Description

Each row of the dataset represents one customer of a subscription service. The columns describe the customer's demographics (age, gender), their relationship with the company (tenure, subscription type, contract length), their behaviour (usage frequency, last interaction, support calls) and their billing

(payment delay, total spend). The target column, Churn, shows whether the customer left the service (1) or stayed (0).

6. Dataset Size

- Raw dataset: 102,828 rows and 12 columns.
- After cleaning: 102,827 rows and 11 columns (one row with a missing target was removed and the CustomerID column was dropped), giving 10 input features and 1 target.
- Train/test split: 80% training (about 82,261 rows) and 20% testing (20,566 rows), with `random_state = 42`.
- Class distribution: about 1,328 retained customers (1.3%) and about 101,499 churned customers (98.7%). The dataset is therefore highly imbalanced.

7. Features/Columns Description

- CustomerID (Integer): unique customer identifier, dropped before training.
- Age (Integer): age of the customer.
- Gender (Categorical): Male or Female.
- Tenure (Integer): number of months the customer has been with the company.
- Usage Frequency (Integer): how often the customer used the service.
- Support Calls (Integer): number of calls made to customer support.

- Payment Delay (Integer): number of days the customer was late in paying.
- Subscription Type (Categorical): Basic, Standard or Premium.
- Contract Length (Categorical): Monthly, Quarterly or Annual.
- Total Spend (Float): total amount spent by the customer.
- Last Interaction (Integer): days since the customer's last interaction.
- Churn (Target): 1 = customer churned, 0 = customer retained.

8. Methodology

The project followed the standard machine learning workflow described below. Tools used: Python, Google Colab, pandas, NumPy, Matplotlib, Seaborn, scikit-learn, joblib, Streamlit, FastAPI and Uvicorn.

8.1 Workflow

1. Load the dataset and inspect its structure.
2. Clean the data (missing values, duplicates, outliers, unnecessary columns).
3. Perform EDA and statistical analysis.
4. Build a preprocessing pipeline.
5. Split the data into training and testing sets.
6. Train four models and evaluate them.

7. Tune the best model with RandomizedSearchCV.
8. Save the final pipeline with joblib.
9. Build the Streamlit dashboard and the FastAPI service.

8.2 Preprocessing Pipeline

A scikit-learn ColumnTransformer was used so that all preprocessing is applied consistently during training and prediction:

- Numerical features (Age, Tenure, Usage Frequency, Support Calls, Payment Delay, Total Spend, Last Interaction): median imputation followed by StandardScaler.
- Categorical features (Gender, Subscription Type, Contract Length): most-frequent imputation followed by OneHotEncoder with handle_unknown set to ignore.

The preprocessing and the model are combined in a single Pipeline, so raw customer data can be passed directly to the saved model.

8.3 Models Trained

Four classification models were trained on the same training data. Class imbalance was handled with class_weight set to balanced for Logistic Regression, Decision Tree and Random Forest, while KNN used distance-based weights.

- Logistic Regression (max_iter = 1000)

- Decision Tree (random_state = 42)
- Random Forest (random_state = 42)
- K-Nearest Neighbors (weights = distance)

Evaluation metrics were Accuracy, Precision, Recall, F1-score, ROC-AUC and the confusion matrix.

8.4 Model Results (before tuning)

- Logistic Regression: Accuracy 0.8723, Precision 0.9992, Recall 0.8714, F1-score 0.9310, ROC-AUC 0.9628.
- Decision Tree: Accuracy 0.9990, Precision 0.9992, Recall 0.9998, F1-score 0.9995, ROC-AUC 0.9676.
- Random Forest: Accuracy 0.9965, Precision 0.9965, Recall 1.0000, F1-score 0.9982, ROC-AUC 0.9997.
- KNN: Accuracy 0.9897, Precision 0.9904, Recall 0.9993, F1-score 0.9948, ROC-AUC 0.8867.

Confusion matrices on the test set, written as: retained correctly identified / retained wrongly predicted as churn / churn wrongly predicted as retained / churn correctly identified.

- Logistic Regression: 234 / 14 / 2,612 / 17,706.
- Decision Tree: 232 / 16 / 4 / 20,314.
- Random Forest: 176 / 72 / 1 / 20,317.
- KNN: 51 / 197 / 15 / 20,303.

8.5 Hyperparameter Tuning

The Random Forest was tuned using RandomizedSearchCV with 5 random combinations (`n_iter = 5`), 3-fold cross-validation (`cv = 3`) and ROC-AUC as the scoring metric. The search space was:

- `n_estimators`: 50, 100, 200, 300
- `max_depth`: None, 10, 20, 30
- `min_samples_split`: 2, 5, 10
- `min_samples_leaf`: 1, 2, 4

Best parameters found: [add the output of `rscv.best_params_` from your notebook].

Random Forest before tuning: Accuracy 0.9965, Precision 0.9965, Recall 1.0000, F1-score 0.9982, ROC-AUC 0.9997.

Random Forest after tuning: Accuracy 0.9979, Precision 0.9983, Recall 0.9996, F1-score 0.9989, ROC-AUC 0.9997.

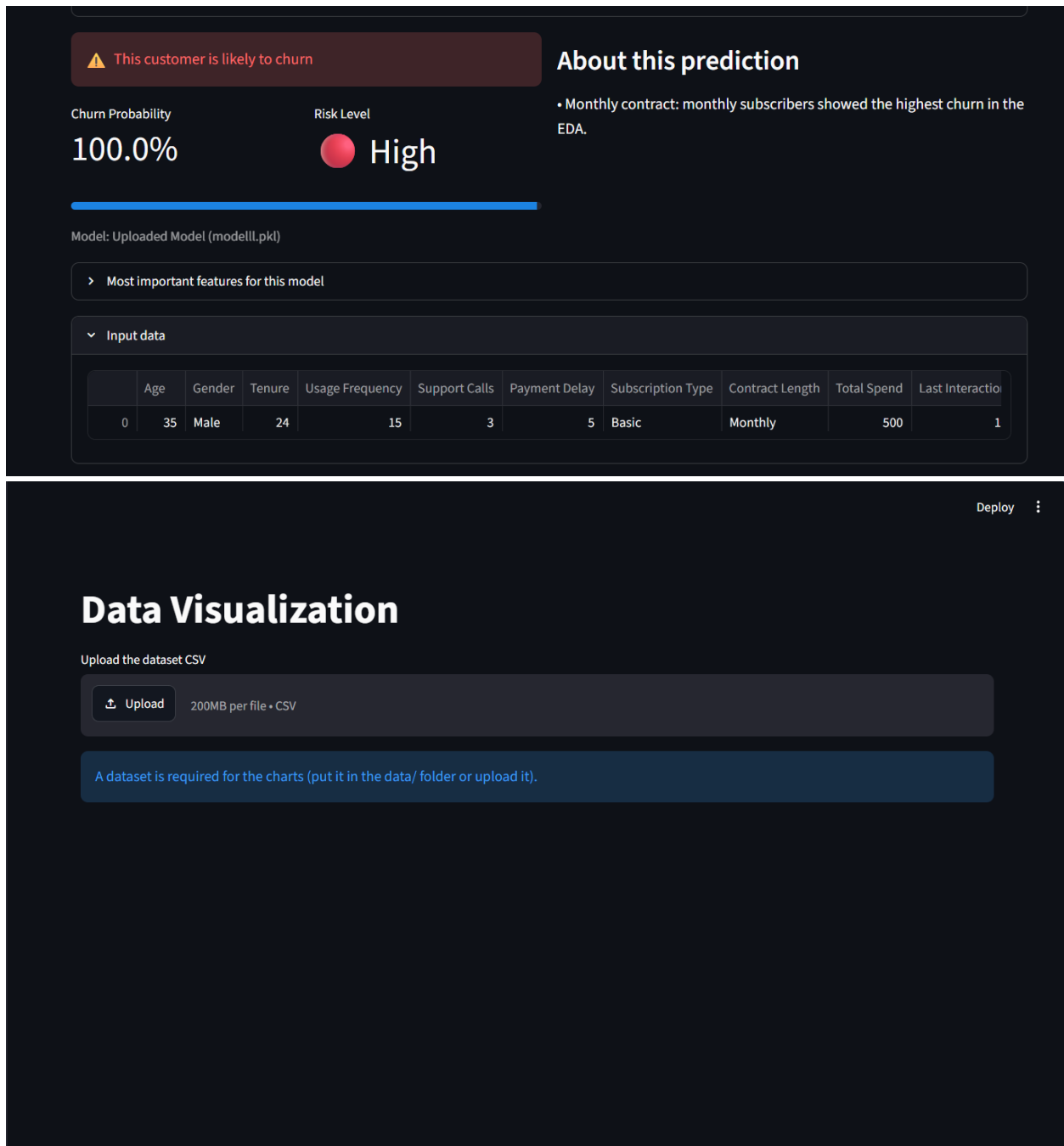
Tuning improved accuracy, precision and F1-score, while ROC-AUC stayed practically unchanged. The tuned pipeline was selected as the final model and saved with joblib as `modelll.pkl`.

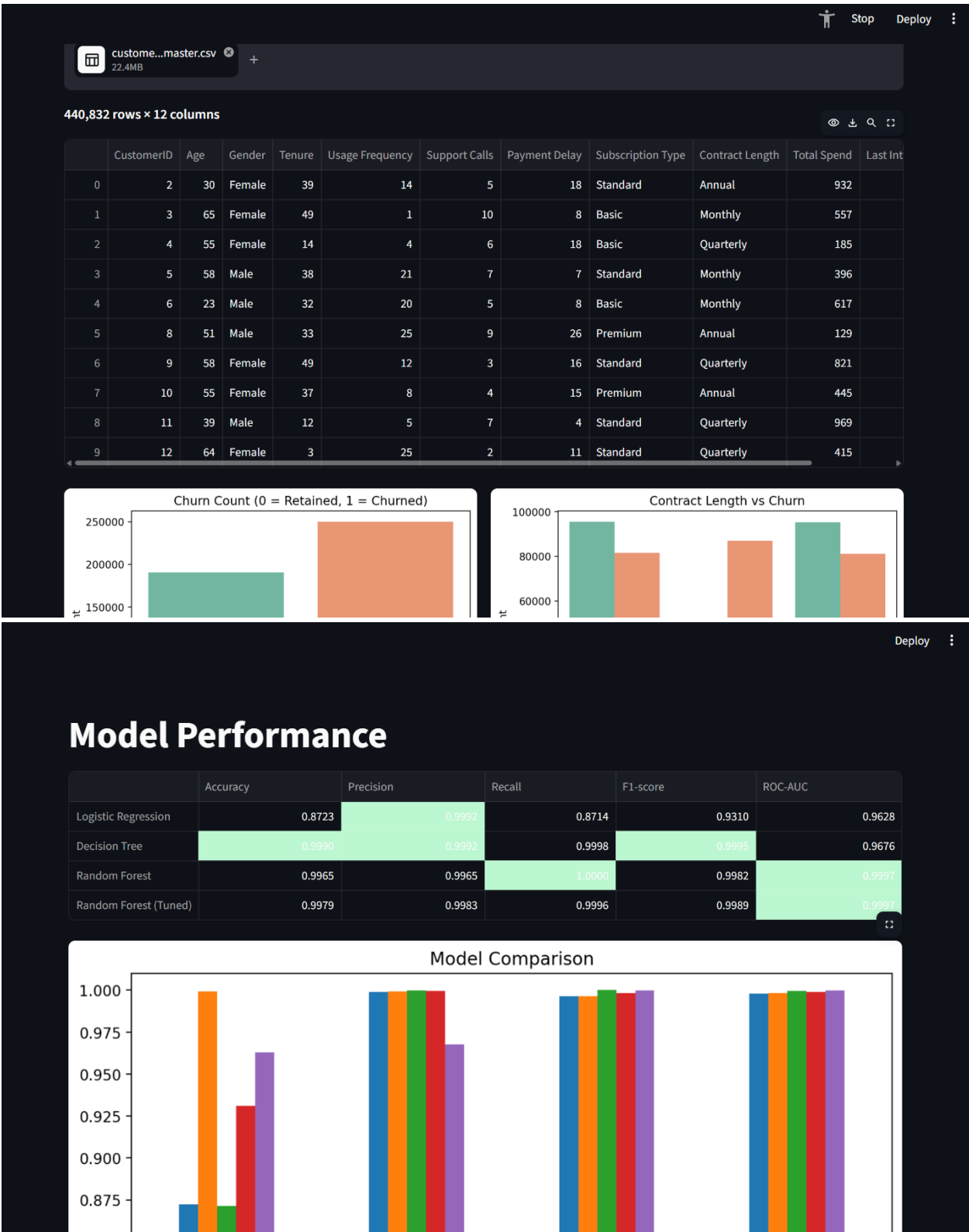
8.6 Streamlit Application

A multi-page Streamlit application (`app.py`) was built with the following pages:

- **Prediction:** the user enters customer details, clicks the Predict button and receives the predicted class, churn probability, risk level (Low, Medium, High), related risk factors and the most important features of the selected model.
- **Data Visualization:** churn count, contract length versus churn, boxplot and histogram of any selected feature, and a correlation heatmap.
- **Model Performance:** comparison table and chart of all models, confusion matrices and feature importance.
- **Multiple model selection:** a sidebar dropdown lets the user switch between the trained models.

The screenshot displays the 'Churn Predictor' application interface. On the left is a dark sidebar with the title 'Churn Predictor' and a 'Navigation' menu containing 'Prediction' (active), 'Data Visualization', and 'Model Performance'. Below the menu is a 'Select model' dropdown showing 'Uploaded Model (model.pkl)'. The main content area has a dark background with the title 'Customer Churn Prediction' and a subtitle 'Enter the customer's details to estimate the risk of churn.' It features a form with several input fields: 'Age' (slider at 35), 'Support Calls' (slider at 3), 'Subscription Type' (dropdown set to 'Basic'), 'Gender' (dropdown set to 'Male'), 'Payment Delay (days)' (slider at 5), 'Contract Length' (dropdown set to 'Monthly'), 'Tenure (months)' (slider at 24), 'Last Interaction (days ago)' (slider at 10), and 'Total Spend' (input field with '500.00' and minus/plus buttons). A large red 'Predict' button is at the bottom of the form. Below the button, a warning message states 'This customer is likely to churn'. At the bottom of the interface, there are labels for 'Churn Probability' and 'Risk Level', and a section titled 'About this prediction' with a bullet point: '• Monthly contract: monthly subscribers showed the highest churn in the'.





8.7 FastAPI Service

The model was also exposed as a REST API (api.py) built with FastAPI and served with Uvicorn. Inputs are validated with Pydantic schemas, and interactive documentation is generated automatically at /docs. The endpoints are:

- POST /predict: predicts churn for one customer, with probability, risk level and risk factors.
- POST /predict/batch: predicts churn for up to 1,000 customers in one request.
- GET /models: lists the available models.
- GET /feature-importance: returns the top features of a model.
- GET /metrics: returns the evaluation metrics of the models.
- GET /health: checks that the service is running.

A model can be chosen with the model query parameter. The API was started with: `python -m uvicorn api:app --reload`

8.8 Deployment

The Streamlit application is deployed online using Streamlit Community Cloud from the GitHub repository.

9. Data Loading

The dataset was loaded into a pandas DataFrame using `pd.read_csv`. It was then inspected with `head(10)`, `tail()`, `shape`, `info()` and columns to understand the number of rows and

columns, the data types and the column names. The raw data has 102,828 rows and 12 columns: 8 numeric columns (including CustomerID and Churn) and 3 text columns.

10. Data Cleaning

- Missing values: one missing value was found in each of Contract Length, Total Spend, Last Interaction and Churn. Numeric columns were filled with the column mean, and the categorical column was filled with the mode.
- Target column: the row with a missing Churn value was dropped, because the target must not be guessed.
- Duplicates: the duplicate check returned 0 duplicated rows.
- Outliers: the IQR method ($1.5 \times \text{IQR}$ rule) found no outliers in any numeric feature. The only flagged column was Churn (1,328 values), which simply reflects the small retained class and is not a data error.
- Unnecessary columns: CustomerID was dropped because it is only an identifier and showed a misleading correlation with churn.
- Inside the pipeline: remaining missing values are handled by median and most-frequent imputers, numeric features are standardized and categorical features are one-hot encoded.

11. Exploratory Data Analysis (EDA)

- Churn distribution: the churn count plot shows a very large difference between the two classes, with only about 1.3% of customers retained.
- Contract length versus churn: the three contract types have similar numbers of customers (roughly 33,000 to 35,000 each). Monthly contracts contain no retained customers at all, while Annual and Quarterly contracts contain only a small number of retained customers.
- Tenure versus churn: both groups span tenures from about 1 to 60 months. The median tenure of retained customers (about 33 months) is slightly higher than that of churned customers (about 30 months).
- Correlation analysis: a heatmap of the numeric features was used to find which features are related to churn (see Statistical Analysis).

12. Statistical Analysis

- Outlier analysis (IQR): no outliers in Age, Tenure, Usage Frequency, Support Calls, Payment Delay, Total Spend or Last Interaction.
- Skewness: all input features have skewness very close to zero (between about -0.04 and 0.03), meaning the distributions are roughly symmetric and no transformation

was needed. The target Churn has a skewness of about - 8.63, which confirms the extreme class imbalance.

- Correlation with Churn: Support Calls 0.13, Payment Delay 0.07, Age 0.05, Last Interaction 0.04, Tenure -0.01, Usage Frequency -0.01 and Total Spend -0.09. CustomerID shows 0.14, but it is only an identifier and was removed.
- Feature correlations with each other are close to zero, so there is no multicollinearity problem.
- Interpretation: linear relationships between single features and churn are weak, which explains why tree-based models perform better than Logistic Regression.
- Baseline check: a model that predicts churn for every customer would already reach about 98.8% accuracy on the test set, so accuracy alone must not be used to judge the models.

13. Data Visualizations

The following visualizations were created with Matplotlib and Seaborn, and the same charts are available on the Data Visualization page of the Streamlit app:

- Count plot of churn status (retained versus churned).
- Grouped count plot of contract length versus churn.
- Box plot of tenure distribution by churn status.
- Correlation heatmap of the numeric features.

- Model comparison chart, confusion matrices and feature importance chart on the Model Performance page.

14. Code Screenshots

OUTLIER DETECTION

```
num_cols = df.select_dtypes(include=['number']).columns
for col in num_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]

    print(f"Outliers in '{col}' are: {len(outliers)}")
```

TRAIN/TEST SPLIT

```
scalar_cols=[ 'Age', 'Tenure', 'Usage Frequency',
               'Support Calls', 'Payment Delay', 'Total Spend', 'Last Interaction']
encoder_cols=['Gender', 'Subscription Type','Contract Length']

x=df.drop(columns=['Churn'])
y=df['Churn']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

[23]

PIPELINE AND COL TRANSFORMER

```
scalar_pipeline=Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scalar',StandardScaler())
])
encoder_pipeline=Pipeline(steps=[
    ('imputer', SimpleImputer(strategy="most_frequent")),
    ('encoder',OneHotEncoder(handle_unknown="ignore"))
])
transformer=ColumnTransformer([
    ('a',scalar_pipeline,scalar_cols),
    ('b',encoder_pipeline,encoder_cols)
])
```

```

models = {
    "Logistic Regression": LogisticRegression(max_iter=1000, class_weight='balanced'),
    "Decision Tree": DecisionTreeClassifier(random_state=42, class_weight='balanced'),
    "Random Forest": RandomForestClassifier(random_state=42, class_weight='balanced'),
    "KNN": KNeighborsClassifier(weights='distance')
}

trained_pipelines = {}
for name, model in models.items():
    print(f" {name}:")
    pipe = Pipeline(steps=[
        ("transformers", transformer),
        ("model", model)
    ])

    pipe.fit(X_train, y_train)
    trained_pipelines[name] = pipe

    y_pred = pipe.predict(X_test)
    y_prob = pipe.predict_proba(X_test)[:, 1]

    print(f"Accuracy : {accuracy_score(y_test, y_pred):}")
    print(f"Precision : {precision_score(y_test, y_pred):}")
    print(f"Recall : {recall_score(y_test, y_pred):}")
    print(f"F1-score : {f1_score(y_test, y_pred):}")
    print(f"ROC-AUC : {roc_auc_score(y_test, y_prob):}")
    print("Confusion Matrix:")
    print(confusion_matrix(y_test, y_pred))

```

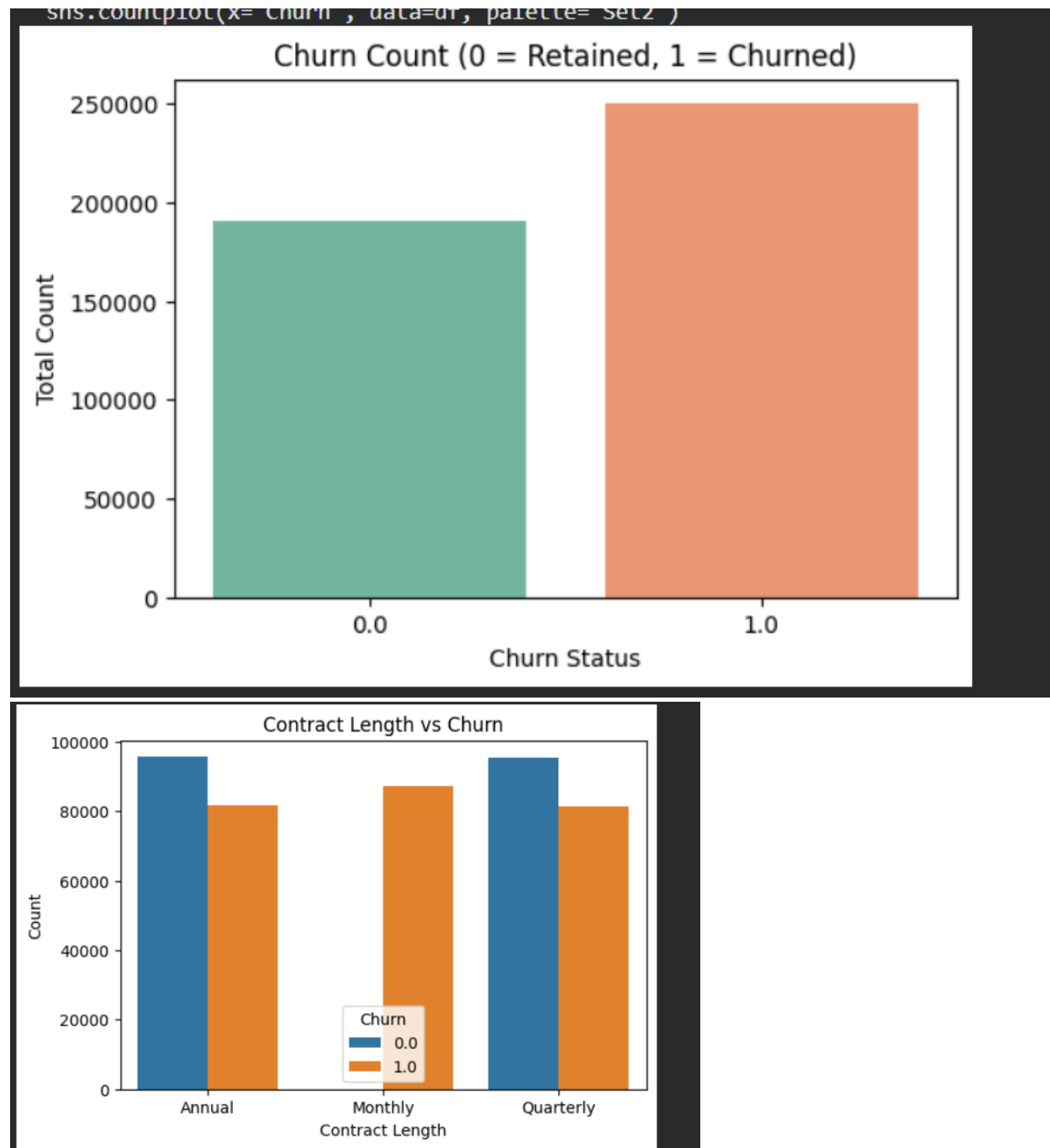
15. Output Screenshots

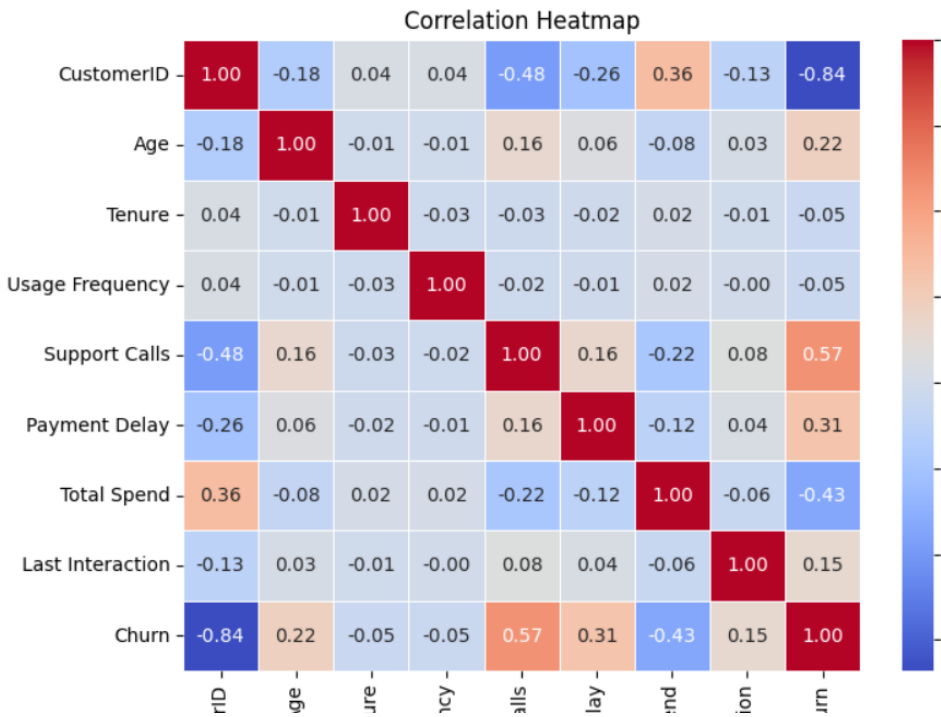
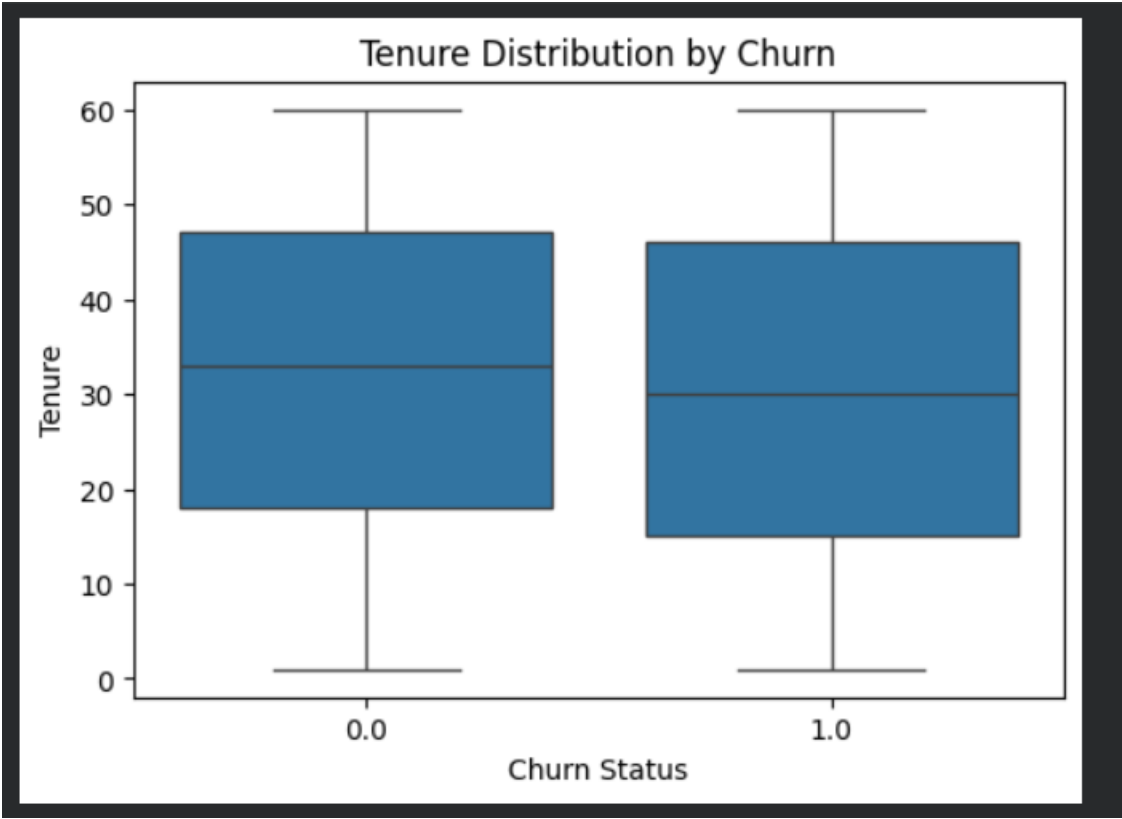
	0
CustomerID	0
Age	0
Gender	0
Tenure	0
Usage Frequency	0
Support Calls	0
Payment Delay	0
Subscription Type	0
Contract Length	0
Total Spend	0
Last Interaction	0
Churn	1

```
Logistic Regression:
Accuracy : 0.8723135271807838
Precision : 0.9992099322799097
Recall : 0.8714440397676937
F1-score : 0.9309637730690362
ROC-AUC : 0.9628485706302056
Confusion Matrix:
[[ 234  14]
 [ 2612 17706]]
Decision Tree:
Accuracy : 0.9990275211514149
Precision : 0.9992129857353664
Recall : 0.9998031302293533
F1-score : 0.9995079708718756
ROC-AUC : 0.9676435005985476
Confusion Matrix:
[[ 232  16]
 [ 4 20314]]
Random Forest:
Accuracy : 0.9964504522026646
Precision : 0.9964686840943646
Recall : 0.9999507825573383
F1-score : 0.9982066966369421
ROC-AUC : 0.9997167020185501
Confusion Matrix:
...
ROC-AUC : 0.8866585801879153
Confusion Matrix:
[[ 51  197]
 [ 15 20303]]
```

```
BEFORE TUNING:
Accuracy : 0.9964504522026646
Precision : 0.9964686840943646
Recall : 0.9999507825573383
F1-score : 0.9982066966369421
ROC-AUC : 0.9997167020185501
AFTER TUNING:
Accuracy : 0.9979091704755422
Precision : 0.9982796755959695
Recall : 0.9996062604587066
F1-score : 0.998942527604948
ROC-AUC : 0.9996953678448158
```

16. Graph Screenshots





17. Major Findings

- The dataset is extremely imbalanced: about 98.7% of the customers are labelled as churned.
- Customers on a Monthly contract show no retained cases in this data, so contract length is the clearest pattern found in the EDA.
- No single numeric feature has a strong linear relationship with churn. Support Calls (positive) and Total Spend (negative) are the most noticeable, but both are weak.
- Tree-based models performed best. Random Forest reached a ROC-AUC of 0.9997, compared with 0.9628 for Logistic Regression and 0.8867 for KNN.
- Hyperparameter tuning raised the Random Forest accuracy from 0.9965 to 0.9979 and the F1-score from 0.9982 to 0.9989.
- Because of the imbalance, the retained class must be checked separately. Logistic Regression and Decision Tree correctly identified most retained customers, while KNN identified only a small part of them.

18. At Least 10 Insights

1. Only about 1.3% of the customers in the dataset are retained, so the data is highly skewed toward churn.

2. A model that always predicts churn would score about 98.8% accuracy, so Precision, Recall, F1-score, ROC-AUC and the confusion matrix are needed for a fair evaluation.
3. Monthly contract customers never appear as retained, which suggests that flexible contracts are linked with the highest churn risk.
4. Annual and Quarterly contracts contain the few retained customers in the dataset, so longer commitments are linked with better retention.
5. The three contract types are almost equal in size, so the contract pattern is not caused by one group being much larger than the others.
6. Retained customers have a slightly higher median tenure (about 33 months) than churned customers (about 30 months), but the tenure ranges overlap almost completely, so tenure is a weak signal.
7. Support Calls has the strongest positive correlation with churn among the features (0.13), suggesting that customers who contact support more often are somewhat more likely to leave.
8. Total Spend has a negative correlation with churn (-0.09), meaning higher-spending customers are slightly more likely to stay.
9. Payment Delay has a small positive correlation with churn (0.07), so late payers are slightly more at risk.
10. Age, Usage Frequency, Tenure and Last Interaction have almost no linear correlation with churn.

11. The numeric features contain no outliers and have near-zero skewness, so no outlier removal or transformation was needed.
12. Logistic Regression had the lowest accuracy (0.8723) but still found 234 of the 248 retained customers in the test set, while Random Forest found 176 and KNN only 51.
13. Decision Tree gave the best accuracy and F1-score, but Random Forest gave the best ROC-AUC (0.9997), meaning it ranks customers by churn risk most reliably.
14. Tuning a Random Forest gave a small but real improvement in accuracy, precision and F1-score without reducing ROC-AUC.

19. Conclusion

This project built a complete churn prediction system, from data cleaning and EDA to model training, tuning and deployment. Four models were compared, and the tuned Random Forest pipeline was selected as the final model because it achieved the highest ROC-AUC (0.9997) together with an F1-score of 0.9989. The model was made available to users through a Streamlit dashboard and a FastAPI service, both of which return the prediction together with its probability.

Limitations: the dataset is heavily imbalanced, with very few retained customers, so the very high scores should be interpreted carefully, especially for the retained class. Future work can include resampling techniques such as SMOTE, decision

threshold tuning, evaluating with precision-recall curves, k-fold cross-validation on the full pipeline, trying gradient boosting models, and adding SHAP explanations.

20. GitHub Repository Link

<https://github.com/HamzaWaseem2005/Customer-Churn-Prediction>